PPS · AMPLIACIÓN — Git y Docker a fondo

Para quien ya ha superado los checkpoints de la sesión. Práctica 1-bis · 1CES CIBER · CIPFP Cheste. Retos independientes: elige por dificultad y hazlos a tu ritmo. ★ suelto · ★★ medio · ★★★ reto.

> **Reglas:** trabaja sobre tu repo `pps-arranque`. No hace falta hacerlos todos ni en orden. Cada reto dice **cómo saber que lo has logrado**. Si te atascas, usa la pista; si sigues atascado, apunta la duda y avanza a otro. **Nunca** subas secretos, no uses `--force`, no hagas público el repo.

Git a fondo

A1 · Deshacer con criterio ★

Haz un commit "malo" a propósito (borra una línea del README). Deshazlo de **tres** formas distintas y entiende la diferencia:

```
git restore README.md          # antes de commit
git revert HEAD                # crea un commit que deshace
git reset --soft HEAD~1        # quita el commit, guarda cambios
```

Pista: `revert` no borra historia (seguro en equipo); `reset` lo mueve (solo en tu rama local).

> Logrado si sabes decir cuándo usar cada uno.

A2 · Guardar sin commit: `stash` ★

Empieza un cambio, guárdalo sin hacer commit, cambia de rama y luego recupéralo.

```
git stash
git switch main
git switch -           # vuelve
git stash pop
```

> Logrado si tu cambio reaparece intacto tras el `pop`.

A3 · Etiqueta versiones (Git ↔ Docker) ★★

Marca una versión en Git y usa **la misma etiqueta** en la imagen Docker.

```
git tag -a v1.0.0 -m "Primera versión del portal"
git push origin v1.0.0
docker build -t pps-web:1.0.0 .
```

> Logrado si `git tag` y `docker image ls` muestran la misma versión.

A4 · Conflicto doble ★★

Provoca un conflicto en **dos ficheros a la vez** (cambia el `<h1>` de `index.html` y una línea del `README.md` en dos ramas). Fusiónalas y resuelve ambos.

Pista: `git status` lista todos los ficheros en conflicto; no olvides `git add` de los dos antes del commit.

> Logrado si `git status` queda limpio y ningún fichero tiene marcadores.

A5 · Investiga la historia ★★

Averigua quién y cuándo cambió una línea, y qué hizo un commit concreto.

```
git blame index.html
git show <hash-del-commit>
git log --oneline --stat
```

> Logrado si localizas el commit que introdujo tu título y explicas qué cambió.

A6 · Un mensaje mejor ★

Corrige el mensaje de tu **último** commit (solo si aún no lo has compartido).

```
git commit --amend -m "Mensaje corregido y claro"
```

> Nunca `--amend` ni `reset` sobre commits ya subidos y compartidos: reescribe historia que otros tienen.

Docker a fondo

B1 · Aprovecha la caché de capas ★★

Cada instrucción del `Dockerfile` es una capa cacheada. Construye dos veces seguidas y compara el tiempo: la segunda reutiliza capas.

```
time docker build -t pps-web:cache .
time docker build -t pps-web:cache .   # mucho más rápida
```

Pista: lo que cambia a menudo (tu `COPY index.html`) va **al final** para no invalidar capas previas.

> Logrado si explicas por qué la segunda build tarda casi nada.

B2 · Chequeo de salud ★★

Añade un `HEALTHCHECK` al `Dockerfile` y observa `docker ps` marcar *healthy*.

```
HEALTHCHECK --interval=10s --timeout=3s \
 CMD python -c "import urllib.request; \
 urllib.request.urlopen('http://localhost:8000')" \
 || exit 1
```

> Logrado si `docker ps` muestra `(healthy)` junto al contenedor.

B3 · Volumen vs COPY (en vivo) ★★

Monta tu página como **volumen** y comprueba que ahora **sí** cambia sin reconstruir:

```
docker run -d --name pps-vol \
 -p 127.0.0.1:8070:8000 \
 -v "$PWD/index.html:/app/index.html" pps-web:v1
```

Edita `index.html`, recarga y verás el cambio al instante.

> Logrado si explicas la diferencia con la lección "la imagen no cambia al editar" y por qué en **producción** se prefiere la imagen (reproducible) y no el volumen.

B4 · Seguridad: no-root + digest fijo ★★★

Comprueba que tu contenedor **no** corre como root y fija la imagen base por **digest** (reproducibilidad).

```
docker exec pps-equipo id        # no debe ser uid=0

# en el Dockerfile, en vez de :slim
FROM python:3.12-slim@sha256:<digest>
```

Pista: obtén el digest con `docker inspect --format='{{index .RepoDigests 0}}' python:3.12-slim`.

> Logrado si `id` no es uid 0 y la build usa un digest fijo. Esto es ya **bastionado**: lo retomaremos en PPS.

B5 · Mirar por dentro ★

Inspecciona el contenedor en marcha: recursos, configuración y procesos.

```
docker stats --no-stream
docker inspect pps-equipo | less
docker top pps-equipo
```

> Logrado si localizas el mapeo de puertos y el usuario dentro de `inspect`.

Entrega (opcional, suma)

Apunta en tu ficha qué retos de ampliación hiciste y, en una frase cada uno, **qué aprendiste**. Los retos ★★★ y la seguridad (B4) cuentan como adelanto de PPS UT2–UT5.